FASE II Y FASE III DOCUMENTACIÓN PROYECTO

GUSTAVO ADOLFO CARDONA BAYONA

JONATHAN SOLER GUEVARA

HUGO ANDRÉS FORERO ANGEL

INGENIERÍA DE SOFTWARE, UNIVERSIDAD MANUELA BELTRÁN

SISTEMAS TRANSACCIONALES A1

JAMILTON FERNANDO BENVIDES

BOGOTÁ D.C.

11 de octubre de 2026

TABLA DE CONTENIDO

1. INFORMACIÓN GENERAL DEL PROYECTO.....3
2. OBJETIVO CENTRAL.....7
3. OBJETIVOS ESPECÍFICOS.....8
4. ARQUITECTURA DE SOFTWARE.....9
5. DIAGRAMA DE CASOS DE USO UML.....14
6. DIAGRAMA DE CLASES UML.....19
7. MODELO RELACIONAL.....23
8. INTEGRACIÓN DE LOS MODELOS.....27
9. REQUERIMIENTOS FUNCIONALES.....30
10. REQUERIMIENTOS NO FUNCIONALES.....32
11. MARCO CONCEPTUAL.....33
12. MARCO CONTEXTUAL.....37
13. MARCO INTEGRADOR: INCORPORACIÓN DEL SERVICIO DE INTELIGENCIA ARTIFICIAL.....40
CONCLUSIONES.....45
REFERENCIAS.....47

1. INFORMACIÓN GENERAL DEL PROYECTO

En esta sección se retoma la contextualización elaborada en la Fase I y se presenta de manera deliberada sin decisiones técnicas: se describe qué problema existe, qué necesidad genera y qué propiedades debe cumplir la solución, en términos del negocio hotelero. Las decisiones sobre cómo se construirá el sistema, es decir, el estilo arquitectónico, las tecnologías, el motor de base de datos y los mecanismos de control de concurrencia, no hacen parte de la contextualización; se adoptan y se justifican a partir de la sección 4, una vez definidos los objetivos y las funcionalidades. Esta separación atiende la observación recibida sobre la Fase I y permite rastrear cada decisión de diseño hasta un elemento concreto del problema.

1.1 NOMBRE DEL SISTEMA

Hotel Reservas: aplicación móvil transaccional (APK) destinada a la administración de reservas de habitaciones en un hotel.

1.2 DESCRIPCIÓN GENERAL

Hotel Reservas consiste en un sistema de procesamiento de transacciones en línea concebido para utilizarse desde dispositivos móviles. La herramienta permitirá al personal de hoteles pequeños y medianos registrar huéspedes y habitaciones, verificar en tiempo real cuáles

se encuentran disponibles, crear, actualizar o cancelar reservas, ingresar pagos, emitir comprobantes, gestionar el ingreso y la salida de los clientes y consultar informes de ocupación. La operación fundamental es la confirmación de una reserva, que debe comportarse como una transacción con propiedades ACID sobre un recurso compartido: la disponibilidad de la habitación. De este modo, dos solicitudes simultáneas para el mismo alojamiento y periodo se resolverán de manera determinista, evitando que se produzcan reservas duplicadas (Elmasri y Navathe, 2016). Esta es una propiedad exigida a la solución y no una decisión de implementación; la manera de garantizarla se define en las secciones 4 y 7.

1.3 PROBLEMÁTICA O NECESIDAD IDENTIFICADA

Durante la Fase I se determinó que muchos hoteles pequeños y medianos administran sus reservas por medios manuales o parcialmente automatizados, entre ellos cuadernos de recepción, archivos compartidos, llamadas telefónicas y aplicaciones de mensajería. Como consecuencia, la información relacionada con cada habitación permanece distribuida entre distintos colaboradores y turnos, sin que exista un repositorio central confiable. Esta dispersión genera cuatro problemas principales: sobreventas o reservas dobles, pérdida de solicitudes, diferencias entre la reserva, el pago y la facturación, y falta de evidencia para atender reclamaciones. Por tanto, se requiere un registro centralizado, seguro y concurrente de la disponibilidad, donde las acciones de confirmar, modificar y cancelar se realicen de forma atómica, aislada y persistente, puedan ejecutarse desde cualquier área del hotel y dejen constancia de cada modificación.

1.4 JUSTIFICACIÓN

La propuesta resulta pertinente porque los inconvenientes identificados producen efectos económicos y reputacionales inmediatos. Una sobreventa puede obligar al hotel a trasladar al

cliente o renunciar al ingreso; una reserva perdida puede dejar desocupada una habitación que habría podido venderse; y una inconsistencia en los registros demanda tiempo adicional para conciliar información. La implementación de un sistema transaccional previene reservas duplicadas, concentra la información en una fuente única accesible desde los dispositivos móviles del personal, conserva el historial de cambios y disminuye las labores manuales. En el ámbito académico, la gestión hotelera constituye un ejemplo claro de procesamiento transaccional, ya que la secuencia de las operaciones y el manejo de la concurrencia repercuten directamente en los resultados económicos. Por ello, el proyecto permite llevar a la práctica los contenidos estudiados en la asignatura.

1.5 ALCANCE DE LA SOLUCIÓN

El diseño contempla un solo hotel, un inventario central de habitaciones, dos perfiles de usuario, recepcionista y administrador, y una habitación vinculada a cada reserva. Cubre el ciclo completo de la reserva, desde la consulta de disponibilidad hasta la salida del huésped, así como los informes básicos de ocupación. Se excluyen la conexión con pasarelas de pago, administradores de canales comerciales y plataformas contables externas; estas integraciones se consideran posibles ampliaciones futuras.

1.6 DELIMITACIÓN DE LAS FASES

La Fase I se limitó a la contextualización: describió la problemática de la gestión fragmentada de la disponibilidad, la necesidad de un registro único y concurrente, la justificación del proyecto y la pregunta problema. Su producto es la comprensión del problema y de las propiedades que debe tener cualquier solución, sin comprometer todavía una tecnología ni un estilo arquitectónico.

La Fase II corresponde al análisis y el diseño. Abarca la formulación de objetivos, el modelo de casos de uso, el diagrama de clases, el modelo relacional, la arquitectura de software y la relación trazable entre todos ellos. Es en esta fase donde se toman las decisiones de diseño y se justifican una a una frente a la problemática: el estilo arquitectónico en la sección 4, la plataforma móvil, el lenguaje y el motor de base de datos en la sección 4.5, y los mecanismos de control de concurrencia en la sección 7.2. En esta fase no se desarrollarán la base de datos, la API ni la aplicación operativa, pues su implementación se reserva para las etapas de construcción y pruebas.

La Fase III amplía el documento maestro con la especificación de requerimientos funcionales y no funcionales, el marco conceptual, el marco contextual y el marco integrador, en el que se propone el servicio de inteligencia artificial que acompañará al sistema. Esta fase mantiene el nivel de análisis y diseño: precisa qué debe hacer el sistema y bajo qué condiciones, pero no implementa la base de datos, la API, la aplicación ni el servicio de inteligencia artificial, cuya construcción y conexión corresponden a las etapas operativas siguientes.

2. OBJETIVO CENTRAL

Diseñar una aplicación móvil transaccional que asegure la integridad, coherencia y trazabilidad de las reservas de habitaciones en hoteles pequeños o medianos. Las operaciones simultáneas de confirmación, modificación y cancelación deberán procesarse con propiedades ACID sobre una fuente central de disponibilidad, con el propósito de evitar las sobreventas y la pérdida de reservas detectadas en el diagnóstico.

3. OBJETIVOS ESPECÍFICOS

En primer lugar, definir y justificar una arquitectura de software que diferencie la interfaz de usuario, las reglas del negocio, la administración de transacciones, el acceso a la información y la base de datos. Así, el procesamiento transaccional permanecerá en un punto central y sus garantías no dependerán del equipo desde el cual se utilice la aplicación.

En segundo lugar, representar mediante casos de uso las funciones disponibles para el recepcionista y el administrador. El modelo debe comprender el registro de huéspedes y habitaciones, la consulta de disponibilidad, todas las etapas de una reserva, el control de entradas y salidas y la generación de informes de ocupación.

En tercer lugar, establecer la estructura estática de la solución a través de un diagrama de clases que incluya las entidades del negocio, sus características, operaciones y vínculos. Se dará especial relevancia al proceso de confirmación de reservas y al registro de auditoría.

En cuarto lugar, formular un modelo relacional con las tablas, claves, restricciones y cardinalidades que eviten reservas confirmadas superpuestas para una misma habitación y permitan gestionar apropiadamente la concurrencia, sobre el motor de base de datos que se seleccione y justifique en la arquitectura.

En quinto lugar, evidenciar la correspondencia entre el problema, los objetivos, las funciones, los casos de uso, las clases, el esquema relacional y la arquitectura, garantizando que todos los componentes del diseño se fundamenten en los elementos previamente definidos.

4. ARQUITECTURA DE SOFTWARE

A partir de esta sección el documento pasa de la contextualización a las decisiones de diseño. Cada decisión se presenta con su justificación y se vincula con el aspecto de la problemática o con el atributo de calidad que la motiva, de modo que sea posible distinguir lo que el problema exige de lo que el equipo eligió para resolverlo. Las decisiones que se toman aquí son cuatro: el estilo arquitectónico, la plataforma de la interfaz de usuario, la tecnología de la lógica de negocio y el motor de base de datos.

4.1 TIPO DE ARQUITECTURA SELECCIONADO

Se propone una arquitectura cliente-servidor estructurada en cuatro niveles: presentación, lógica de negocio, acceso a datos y almacenamiento. La presentación se ejecutará en el dispositivo móvil, mientras que los demás niveles funcionarán en el servidor. Cada capa se relacionará únicamente con la que se encuentra inmediatamente debajo mediante interfaces establecidas. Esta organización coincide con el patrón descrito por Bass et al. (2021), que limita las dependencias entre módulos a relaciones autorizadas entre capas consecutivas. En la presentación se empleará además el patrón MVVM Modelo, Vista y Modelo de Vista, recomendado por Microsoft (2024b) para el desarrollo con .NET MAUI.

4.2 DIAGRAMA DE LA ARQUITECTURA

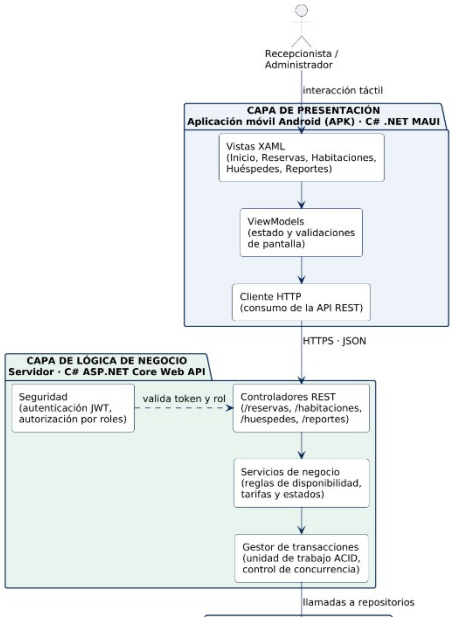


Figura 1 Arquitectura en capas del sistema Hotel Reservas

Nota. Creación propia. Las flechas muestran el sentido de las solicitudes y cada nivel depende exclusivamente del que se encuentra debajo.

4.3 COMPONENTES PRINCIPALES Y FUNCIÓN DE CADA UNO

La capa de presentación corresponde a la aplicación Android programada en C# mediante .NET MAUI y generada como archivo APK. Las vistas XAML representan las pantallas de inicio, reservas, habitaciones, huéspedes e informes disponibles en el repositorio del proyecto. Los modelos de vista administran el estado de las pantallas y validan el formato de la información antes de enviarla, mientras que el cliente HTTP centraliza el acceso a la API REST. Este nivel no implementa reglas del negocio ni se conecta directamente con la base de datos.

La lógica de negocio se implementará mediante una API REST creada en C# con ASP.NET Core Web API. Sus controladores publicarán los recursos de reservas, habitaciones, huéspedes, pagos e informes y transformarán las solicitudes HTTP en invocaciones a los servicios. Estos servicios contendrán las reglas del dominio, como el cálculo de la disponibilidad, la selección de tarifas y las transiciones válidas entre estados. A su vez, el gestor transaccional reunirá en una sola operación acciones como validar nuevamente la disponibilidad, crear la

reserva, registrar el pago y expedir el comprobante, aplicando el control de concurrencia dentro de una única transacción de base de datos. La seguridad validará la identidad mediante tokens JWT y limitará las acciones según el perfil asignado.

Para la capa de acceso a datos se utilizará Entity Framework Core como herramienta de mapeo objeto-relacional. Los repositorios proporcionarán las operaciones de consulta y almacenamiento de cada entidad, y DbContext controlará la correspondencia entre objetos y tablas, la conexión y el alcance de las transacciones. En situaciones que exijan un manejo más específico, como la utilización de sugerencias de bloqueo, se empleará ADO.NET junto con instrucciones T-SQL expresas.

La capa de almacenamiento estará conformada por una base de datos Microsoft SQL Server gestionada desde SQL Server Management Studio. Allí se ubicarán las tablas, claves, reglas de integridad, índices y registros de auditoría. Las transacciones ofrecidas por el motor garantizarán en última instancia el cumplimiento de las propiedades ACID (Microsoft, 2024a).

4.4 COMUNICACIÓN ENTRE COMPONENTES

La aplicación se conectará con la API por medio de solicitudes HTTPS cuyos contenidos se enviarán en JSON, conforme al enfoque REST. Cada recurso dispondrá de una ruta y se operará mediante los métodos GET, POST, PUT y DELETE. Las respuestas incluirán estados HTTP interpretados por la aplicación; por ejemplo, si se intenta asignar una habitación que acaba de ser reservada, se devolverá el código 409 conflicto acompañado de una explicación. En el servidor, los controladores llamarán a los servicios y estos al gestor transaccional mediante métodos directos e inyección de dependencias. El nivel de datos se comunicará con SQL Server a través del proveedor de .NET, que remitirá instrucciones T-SQL usando el protocolo TDS. La

unidad de trabajo comenzará con BEGIN TRANSACTION y terminará con COMMIT o ROLLBACK, según el resultado obtenido.

4.5 JUSTIFICACIÓN DE LA ARQUITECTURA Y DE LAS TECNOLOGÍAS

La organización por capas responde a la necesidad detectada porque sitúa el procesamiento transaccional en el servidor y en la base de datos. En consecuencia, todos los teléfonos consultan y actualizan el mismo registro de disponibilidad, superando la dispersión que ocasiona las reservas dobles. Esta separación también permite modificar o ampliar la interfaz móvil sin cambiar las reglas del negocio, habilitar en el futuro el consumo de la API desde un portal web o un gestor de canales y sustituir o actualizar el motor de datos sin alterar la aplicación. En cuanto a calidad, facilita la modificación y las pruebas aisladas de los componentes, y fortalece la seguridad al centralizar la autenticación y la autorización en un único acceso (Bass et al., 2021). A diferencia de una solución con almacenamiento local, conserva la interacción concurrente entre usuarios, requisito esencial del proyecto.

Las decisiones tecnológicas se justifican de la misma manera. La interfaz se construye como aplicación móvil Android, distribuida en formato APK, porque la necesidad establece que el personal debe consultar y confirmar disponibilidad desde el lugar del hotel donde se encuentre, y porque el docente de la asignatura recomendó esta modalidad; se descartó una base de datos embebida en el dispositivo (SQLite) porque rompería el registro único y, con él, la garantía frente a la doble reserva. Se elige C# con .NET MAUI para la aplicación y C# con ASP.NET Core Web API para la lógica de negocio porque permiten usar un solo lenguaje en las capas de presentación y de servidor, cuentan con soporte del fabricante y se integran de forma nativa con el motor de datos elegido. Se elige Microsoft SQL Server porque ofrece transacciones explícitas, niveles de aislamiento configurables, bloqueos de fila y el tipo ROWVERSION, que

son precisamente los mecanismos que la sección 7.2 emplea para impedir el solapamiento de reservas (Microsoft, 2024a); se descartó MySQL por decisión del equipo y se consideró PostgreSQL como alternativa equivalente, sin ventaja decisiva para este alcance. Ninguna de estas decisiones altera la contextualización de la Fase I; todas responden a ella.

5. DIAGRAMA DE CASOS DE USO UML

5.1 ACTORES

Pressman y Maxim (2021) señalan que un caso de uso explica la manera en que un actor interactúa con el sistema para lograr un propósito; por ello, el modelado comienza con la identificación de quienes participan. La propuesta contempla dos actores humanos. El recepcionista desempeña las tareas operativas: atender clientes, verificar disponibilidad, crear, modificar o cancelar reservas, registrar pagos y controlar el ingreso y la salida. El administrador mantiene el inventario de habitaciones y tarifas, gestiona las cuentas de usuario y consulta los informes de ocupación. Los dos perfiles deben autenticarse antes de acceder a las funciones. No se incorporan actores externos porque esta fase no considera servicios de pago ni plataformas de terceros.

5.2 DIAGRAMA

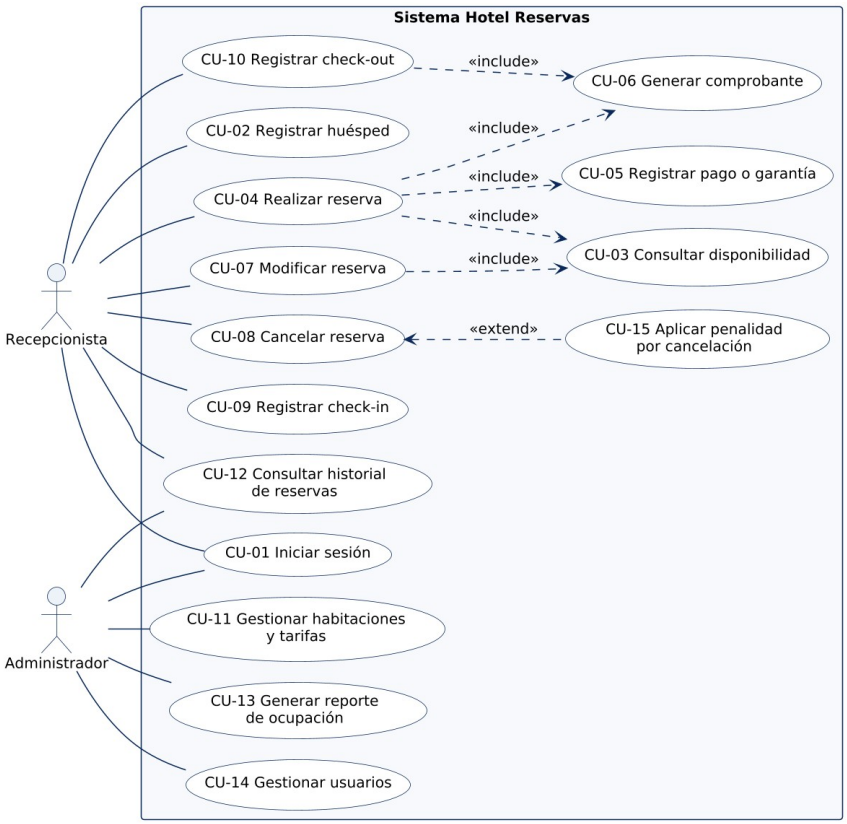


Figura 2 Diagrama de casos de uso del sistema Hotel Reservas

Nota. Diseño propio basado en UML 2.5 (Object Management Group, 2017). El contorno rectangular delimita el sistema; «include» representa una función obligatoria que se reutiliza y «extend» identifica un comportamiento opcional condicionado.

CU-04 y CU-07 incorporan CU-03, Consultar disponibilidad porque una reserva no puede registrarse ni modificarse sin comprobar previamente que la habitación esté libre durante el periodo seleccionado. Esta consulta también puede ser ejecutada directamente por el recepcionista. CU-04 incluye CU-05 y CU-06, puesto que la confirmación requiere registrar el pago o la garantía y producir el comprobante como parte de una misma transacción. Por otra parte, CU-15 amplía CU-08 únicamente cuando la cancelación se presenta después del plazo admitido por las políticas del establecimiento.

5.3 DESCRIPCIÓN DE LOS PRINCIPALES CASOS DE USO

Nombre: CU-04 Crear una reserva.

Actor principal: Recepcionista.

Objetivo: Formalizar la reserva de una habitación para un huésped durante unas fechas determinadas, sin permitir que el mismo espacio sea asignado a otra solicitud.

Precondiciones: El recepcionista se encuentra autenticado. El huésped ya está registrado o se incorpora durante el proceso mediante CU-02. Debe existir, como mínimo, una habitación perteneciente a la categoría requerida.

Flujo principal: El recepcionista especifica la categoría de habitación, las fechas de llegada y salida y la cantidad de personas. La plataforma ejecuta CU-03, presenta las alternativas

libres y muestra sus tarifas. Después de elegir una habitación y vincular al huésped, el sistema abre la transacción, vuelve a comprobar la disponibilidad con bloqueo, genera la reserva con estado confirmada, registra el pago o la garantía por medio de CU-05, produce el comprobante en CU-06, actualiza la bitácora y confirma la operación. Finalmente, informa el código asignado y presenta el comprobante.

Flujo alternativo: Cuando la segunda validación detecta que otra transacción ya ocupó la habitación, todas las acciones se revierten y se informa “habitación no disponible”, sin almacenar datos incompletos.

Resultado esperado: La reserva queda confirmada, vinculada con el pago y el comprobante, almacenada como una unidad indivisible y disponible inmediatamente para todos los usuarios.

Nombre: CU-03 Consultar disponibilidad.

Actor principal: Recepcionista.

Objetivo: Identificar las habitaciones disponibles de una categoría determinada dentro de un intervalo de fechas.

Precondiciones: El usuario inició sesión, las fechas ingresadas son correctas y la salida se programó después de la entrada.

Flujo principal: El usuario selecciona el tipo de habitación y el periodo. El sistema localiza aquellas unidades que no estén en mantenimiento ni tengan reservas confirmadas o huéspedes alojados en fechas que se superpongan con la consulta. Luego presenta los resultados, la tarifa nocturna aplicable y el costo total aproximado.

Resultado esperado: Se obtiene una relación de habitaciones libres calculada con datos del repositorio central, no con registros almacenados localmente.

Nombre: CU-08 Cancelar reserva.

Actor principal: Recepcionista.

Objetivo: Dejar sin efecto una reserva confirmada y devolver la habitación al inventario disponible.

Precondiciones: La reserva está registrada y conserva el estado confirmado. El recepcionista se encuentra autenticado.

Flujo principal: El recepcionista localiza la reserva mediante su código o los datos del cliente y registra la causa de la cancelación. El sistema revisa la política aplicable y, si corresponde, activa CU-15 para calcular la penalidad. Posteriormente abre una transacción, cambia la reserva a cancelada, registra la devolución completa o parcial del pago, actualiza la bitácora y confirma los cambios. Las fechas asociadas a la habitación quedan nuevamente disponibles.

Resultado esperado: La reserva figura como cancelada, el pago se revierte conforme a la política y la habitación se libera. El historial conserva la identidad del responsable y el momento de la operación.

Nombre: CU-09 Registrar check-in.

Actor principal: Recepcionista.

Objetivo: Confirmar la llegada del cliente y registrar que la habitación se encuentra efectivamente ocupada.

Precondiciones: Hay una reserva confirmada cuya entrada corresponde al día en curso y la habitación aparece con estado reservada.

Flujo principal: El recepcionista consulta la reserva, valida la identidad del huésped y confirma su ingreso. Dentro de una sola transacción, el sistema cambia la reserva al estado en casa, marca la habitación como ocupada y guarda el movimiento en la bitácora.

Resultado esperado: La reserva queda en estado en casa, la habitación se registra como ocupada y se almacena la hora de llegada.

Nombre: CU-13 Generar reporte de ocupación.

Actor principal: Administrador.

Objetivo: Consultar la ocupación efectiva y estimada del establecimiento durante un intervalo determinado.

Precondiciones: El administrador está autenticado y existen reservas en el sistema.

Flujo principal: El administrador define el periodo de análisis. Para cada fecha, el sistema determina cuántas habitaciones están ocupadas, reservadas o disponibles y calcula el porcentaje de ocupación utilizando las reservas confirmadas, en casa y cerradas. Los resultados se visualizan en pantalla.

Resultado esperado: Se genera un informe de ocupación acorde con la situación real de las reservas, sin necesidad de realizar conciliaciones manuales.

6. DIAGRAMA DE CLASES UML

El modelo de clases muestra la organización estática necesaria para ejecutar los casos de uso definidos. Sus entidades representan los conceptos identificados en esos procesos: huésped, habitación, categoría de habitación, tarifa, reserva, pago, comprobante y bitácora. Usuario se plantea como clase abstracta y se especializa en Recepcionista y Administrador; además de representar los perfiles del sistema, permite reconocer al responsable de cada acción. ServicioReservas corresponde a la lógica de negocio y no a una entidad del dominio. Se incorpora porque coordina la operación crítica de confirmación.

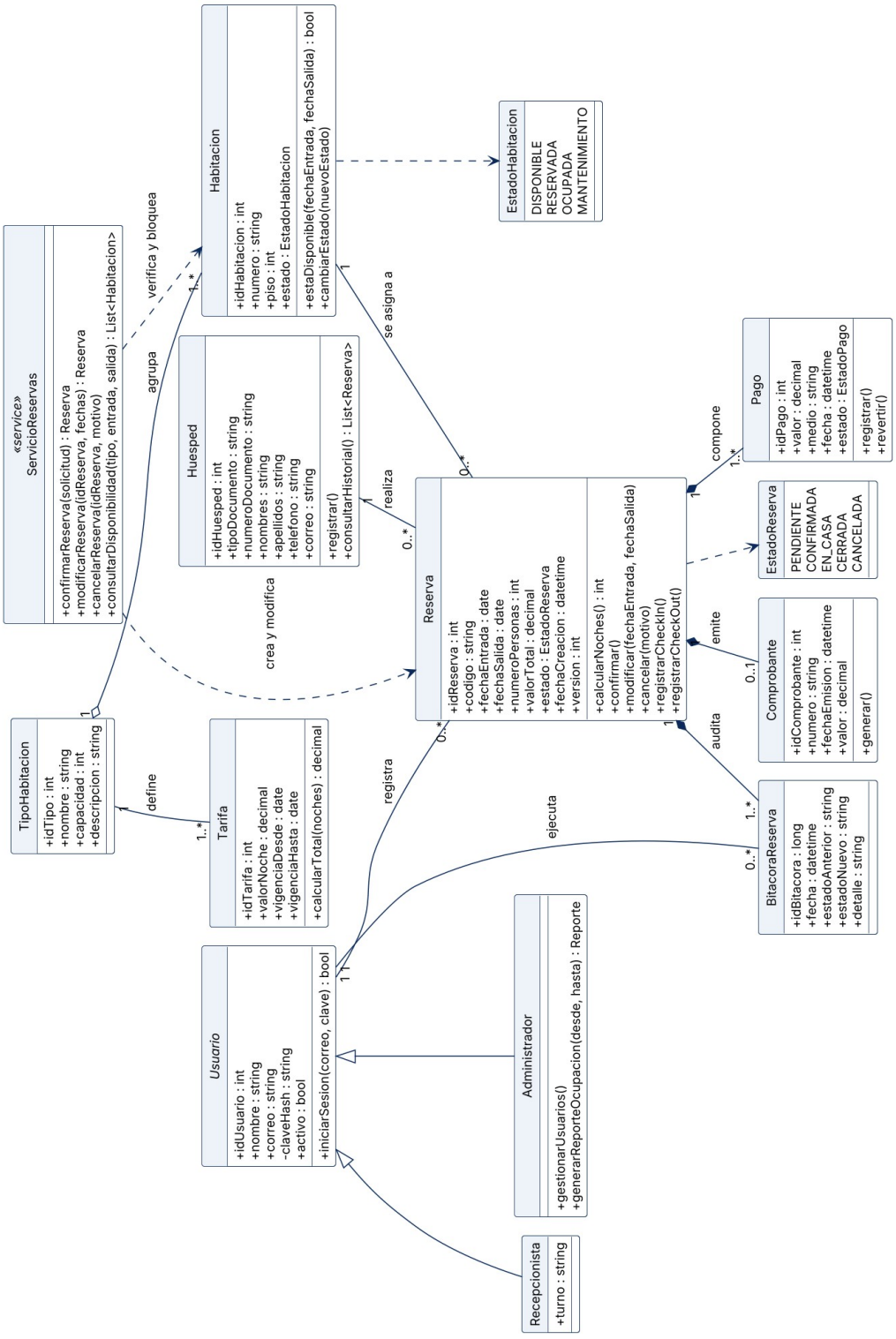


Figura 3 Diagrama de clases del sistema Hotel Reservas

Nota. Elaboración propia con UML 2.5 (Object Management Group, 2017). El diamante sólido expresa composición, el diamante sin relleno representa agregación, las flechas con extremo vacío señalan herencia y los trazos discontinuos indican dependencia.

6.1 RELACIONES Y MULTIPLICIDADES

Cada tipo reúne una o varias habitaciones y dispone de una o más tarifas con periodos definidos. La asociación se representa como agregación, pues las habitaciones pueden existir aunque se reorganice su clasificación. Un huésped puede no tener reservas o registrar varias, mientras que toda reserva debe corresponder a un único huésped. Una habitación admite múltiples reservas en distintos momentos, pero cada reserva se asigna exactamente a una habitación. Cada usuario puede registrar ninguna o muchas reservas, lo que permite establecer quién creó cada registro, y de la misma forma ejecuta ninguna o muchas anotaciones de bitácora, de modo que el responsable de cada cambio de estado queda identificado. Estas multiplicidades coinciden con las cardinalidades del modelo relacional de la sección 7: 1 a 1..N entre tipo y habitaciones y entre tipo y tarifas; 1 a 0..N entre huésped, habitación y usuario respecto de reserva, y entre usuario y bitácora; 1 a 1..N entre reserva y pagos y entre reserva y bitácora; y 1 a 0..1 entre reserva y comprobante.

La composición indica que el pago, el comprobante y la bitácora dependen de la reserva y carecen de sentido sin ella, por lo que desaparecen cuando esta se elimina. Toda reserva contará al menos con un pago la garantía inicial y podrá acumular otros movimientos, como abonos, saldos o devoluciones. También podrá tener un solo comprobante como máximo y deberá incluir al menos una anotación de bitácora correspondiente a su creación. La generalización de Usuario en Recepcionista y Administrador permite reutilizar el mecanismo de autenticación y, al mismo tiempo, distinguir los permisos de cada perfil.

6.2 CORRESPONDENCIA CON LOS CASOS DE USO

CU-04 se materializa mediante confirmarReserva, método de ServicioReservas que consulta estaDisponible en Habitacion, crea la Reserva, registra el Pago, produce el Comprobante y añade la entrada en BitacoraReserva. CU-03 se relaciona con consultarDisponibilidad, operación que revisa las habitaciones de una categoría y aplica estaDisponible. CU-08 se ejecuta con cancelarReserva, que utiliza cancelar en Reserva y revertir en Pago. CU-09 y CU-10 corresponden respectivamente a registrarCheckIn y registrarCheckOut de Reserva, y ambos actualizan Habitacion mediante cambiarEstado. CU-13 se vincula con generarReporteOcupacion de Administrador. Finalmente, el campo version de Reserva contribuye al control de concurrencia optimista detallado en el esquema relacional.

7. MODELO RELACIONAL

El esquema relacional convierte las clases persistentes en tablas de Microsoft SQL Server, transforma sus atributos en columnas con tipos de datos definidos y ubica las claves foráneas en el extremo de multiplicidad “muchos” de cada relación, de acuerdo con las reglas propuestas por Elmasri y Navathe (2016). La jerarquía de Usuario se almacena en una sola tabla denominada USUARIO, acompañada por una columna rol, debido a que sus especializaciones incorporan pocos atributos adicionales. Las enumeraciones se guardan como cadenas breves sujetas a validaciones CHECK. ServicioReservas no se convierte en tabla porque pertenece a la lógica de negocio.

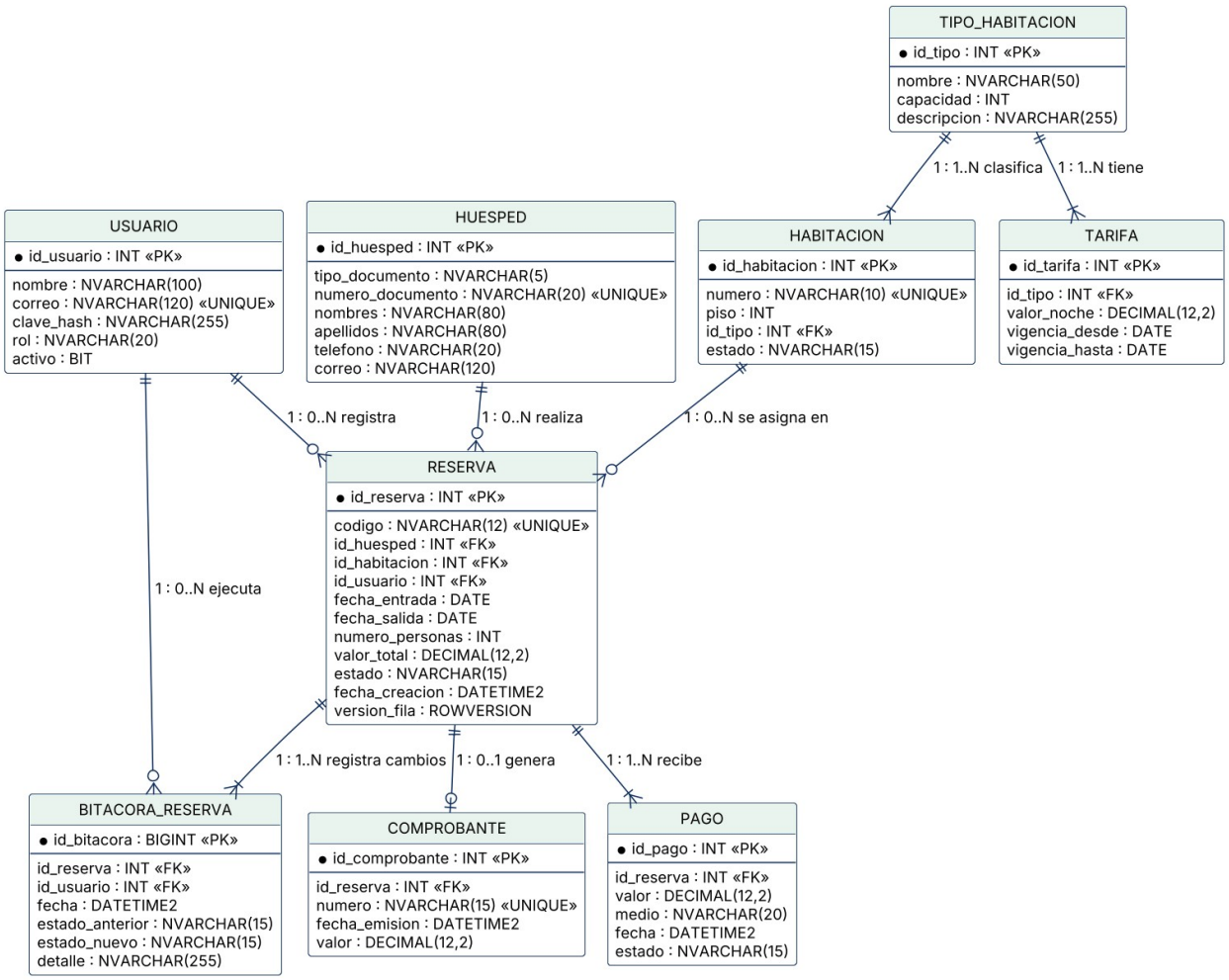


Figura 4 Modelo relacional del sistema Hotel Reservas

Nota. Construcción propia mediante notación de pata de gallo. PK corresponde a clave primaria, FK a clave foránea y UNIQUE a una condición de valor único. Cada etiqueta señala la cardinalidad mínima y máxima de la relación (1 : 0..N, 1 : 1..N o 1 : 0..1), con los mismos valores que las multiplicidades de la Figura 3 y de la sección 6.1.

7.1 DESCRIPCIÓN DE LAS TABLAS

USUARIO conserva las credenciales del personal, su perfil RECEPCIONISTA o ADMINISTRADOR, el correo irrepetible y la contraseña protegida mediante hash. HUESPED reúne los datos de identificación y contacto de los clientes y evita documentos duplicados. TIPO_HABITACION organiza las unidades por denominación y capacidad; TARIFA guarda el precio nocturno de cada categoría y su periodo de aplicación, de manera que se mantenga el historial de valores. HABITACION representa el inventario físico e incluye número único, piso, tipo y situación actual.

RESERVA constituye la tabla principal. Contiene referencias al huésped, la habitación y el usuario responsable, así como las fechas de llegada y salida, la cantidad de ocupantes, el importe calculado con la tarifa vigente, el estado y el momento de creación. SQL Server modifica automáticamente version_fila, columna ROWVERSION utilizada para la concurrencia optimista: el sistema remite la versión previamente consultada y solo permite la actualización cuando todavía coincide con la almacenada. PAGO almacena cada transacción monetaria de la reserva, junto con su forma de pago y condición registrado o revertido. COMPROBANTE conserva el soporte generado al confirmar, identificado con numeración única. BITACORA_RESERVA documenta cada cambio de estado, el usuario que lo efectuó, la fecha y los estados anterior y posterior, aportando la evidencia necesaria frente a posibles reclamaciones.

7.2 RESTRICCIONES QUE GARANTIZAN LA INTEGRIDAD TRANSACCIONAL

Las claves foráneas evitan crear reservas sin huésped, habitación o usuario, y también impiden pagos, comprobantes o movimientos de bitácora desvinculados de una reserva. Mediante restricciones CHECK se valida que la salida sea posterior a la entrada, que la cantidad de personas no exceda la capacidad establecida y que el estado pertenezca al conjunto

autorizado. La regla principal, evitar dos reservas activas superpuestas para la misma habitación, se respalda con un índice sobre (id_habitacion, fecha_entrada, fecha_salida) y una comprobación ejecutada dentro de la transacción de confirmación. Para ello se utiliza el aislamiento SERIALIZABLE o las sugerencias UPDLOCK y HOLDLOCK sobre las filas correspondientes, haciendo que dos operaciones simultáneas sobre la misma habitación se atiendan secuencialmente y solo una sea confirmada (Microsoft, 2024a). El fragmento siguiente ejemplifica la transacción prevista para la etapa de construcción.

8. INTEGRACIÓN DE LOS MODELOS

La documentación se desarrolló siguiendo una secuencia coherente. La dispersión de los datos y las reservas duplicadas identificadas en el problema condujeron al objetivo general de asegurar integridad, consistencia y trazabilidad en condiciones de concurrencia. Posteriormente, este propósito se dividió en objetivos específicos asociados con la arquitectura, los casos de uso, las clases, el esquema relacional y la trazabilidad. Cada función fue representada por un caso de uso; cada caso se sustenta en determinadas clases; cada clase persistente se refleja en una tabla; y la arquitectura establece el componente donde debe procesarse cada acción. La Tabla 1 sintetiza estas relaciones para las funciones más importantes.

Tabla 1 Matriz de trazabilidad entre problemática, objetivos, casos de uso, clases, tablas y componentes

Problema	Objetivo	Caso de uso	Clases	Tablas	Componente
Reservas duplicadas por validación sin control	1, 3, 4	CU-03, CU-04	ServicioReservas, Habitacion, Reserva, Tarifa	RESERVA, HABITACION, TARIFA	Gestión transaccional y base de datos mediante aislamiento y bloqueos
Diferencias entre reserva, pago y soporte	3, 4	CU-04, CU-05, CU-06	Reserva, Pago, Comprobante	RESERVA, PAGO, COMPROBANTE	Gestor transaccional como unidad de

					trabajo indivisible
Pérdida o modificación de reservas sin seguimiento	2, 3	CU-07, CU-08, CU-15	Reserva, Pago, ServicioReservas	RESERVA, PAGO	Servicios del negocio y repositorios
Información desactualizada sobre las habitaciones	2, 3	CU-09, CU-10, CU-11	Habitacion, Reserva, TipoHabitacion	HABITACION, RESERVA, TIPO_HABITACION	Servicios del negocio y DbContext
Ausencia de evidencia para atender reclamaciones	3, 4	CU-04, CU-07, CU-08, CU-09, CU-10	BitacoraReserva, Usuario	BITACORA_RESERVA, USUARIO	Gestor transaccional y seguridad para identificar al usuario
Datos dispersos para apoyar decisiones	2	CU-12, CU-13	Administrador, Reserva, Huesped	RESERVA, HABITACION, HUESPED	Controladores y servicios destinados a reportes
Acceso a los datos sin autorización suficiente	1, 2	CU-01, CU-14	Usuario, Recepcionista, Administrador	USUARIO	Seguridad basada en JWT y perfiles

Nota. Elaboración propia. La numeración de los objetivos específicos sigue el orden presentado en el apartado 3.

La matriz confirma que los componentes fueron concebidos de forma integrada. La confirmación transaccional, respuesta principal ante las reservas dobles, se identifica como CU-04; se coordina desde ServicioReservas usando Reserva, Pago y Comprobante; se guarda en las tablas equivalentes bajo las reglas expuestas en el apartado 7.2; y se procesa en el gestor transaccional de la lógica de negocio con el respaldo del control de concurrencia del motor de datos. De manera semejante, la necesidad de seguimiento se concreta en la clase y la tabla de bitácora, mientras que el módulo de seguridad permite reconocer al usuario responsable de cada movimiento.

Como cierre de la integración conviene precisar qué queda definido en esta fase y qué se traslada a la siguiente. Quedan definidos de manera estable el objetivo central y los objetivos específicos, los dos actores y los quince casos de uso, las clases del dominio con sus multiplicidades, las nueve tablas con sus claves y cardinalidades, el estilo arquitectónico en capas y la trazabilidad entre todos estos elementos. Pasan a la Fase III, que se desarrolla en las secciones 9 a 13, la especificación de los requerimientos funcionales y no funcionales que harán verificable este diseño, la fundamentación conceptual y contextual, y la propuesta del servicio de inteligencia artificial. La elección final entre bloqueo pesimista y concurrencia optimista, así como los scripts, la API y la aplicación operativa, se reservan para las etapas de construcción y pruebas.

9. REQUERIMIENTOS FUNCIONALES

Los requerimientos funcionales describen las acciones que el sistema transaccional debe permitir a sus usuarios. Siguiendo a Sommerville (2011), se formulan como declaraciones de servicios que el sistema ofrece, con el actor que las solicita y el resultado observable que producen, de manera que cada una pueda convertirse después en un caso de prueba. La Tabla 2 presenta los quince requerimientos funcionales del proyecto. Su numeración se corresponde uno a uno con los casos de uso de la sección 5, lo que conserva la trazabilidad construida en la Fase II y evita que aparezcan funciones no respaldadas por el diseño.

Tabla 2 Requerimientos funcionales del sistema Hotel Reservas

Código	Nombre	Descripción	Actor	Resultado esperado
RF01	Autenticar usuario	Permitir el inicio de sesión con correo y contraseña y asignar las funciones disponibles según el perfil (recepcionista o administrador).	Recepcionista, administrador	Sesión iniciada con un token válido y un menú acorde con el perfil (CU-01).
RF02	Registrar huésped	Capturar tipo y número de documento, nombres, apellidos, teléfono y correo del huésped, impidiendo documentos duplicados.	Recepcionista	Huésped almacenado con identificador único y disponible para asociarse a reservas (CU-02).
RF03	Consultar disponibilidad	Mostrar las habitaciones libres de un tipo para un intervalo de fechas, excluyendo las que estén en mantenimiento o con reservas activas solapadas.	Recepcionista	Lista de habitaciones disponibles con tarifa por noche y valor estimado (CU-03).
RF04	Crear reserva	Registrar una reserva para un huésped, una habitación y un periodo, verificando de nuevo la disponibilidad dentro de la misma transacción.	Recepcionista	Reserva confirmada con código único, o rechazo con el mensaje de habitación no disponible, sin datos parciales (CU-04).
RF05	Registrar pago o garantía	Asociar a la reserva el valor pagado, el medio de pago y la fecha, como parte de la confirmación o como abono posterior.	Recepcionista	Pago registrado y vinculado a la reserva (CU-05).
RF06	Generar comprobante	Emitir el soporte de la reserva con numeración única, datos del huésped, habitación, fechas y valor.	Recepcionista	Comprobante generado una sola vez por reserva y consultable desde la aplicación (CU-06).

Código	Nombre	Descripción	Actor	Resultado esperado
RF07	Modificar reserva	Cambiar fechas, habitación o número de personas de una reserva confirmada, validando la nueva disponibilidad.	Recepcionista	Reserva actualizada y registro del cambio en la bitácora (CU-07).
RF08	Cancelar reserva	Dejar sin efecto una reserva confirmada, registrar el motivo y liberar la habitación para las fechas asociadas.	Recepcionista	Reserva en estado cancelada, pago revertido según la política y habitación liberada (CU-08).
RF09	Registrar check-in	Confirmar la llegada del huésped y marcar la habitación como ocupada.	Recepcionista	Reserva en estado en casa y habitación ocupada (CU-09).
RF10	Registrar check-out	Cerrar la estadía, registrar el saldo final y devolver la habitación al inventario disponible.	Recepcionista	Reserva cerrada y habitación disponible (CU-10).
RF11	Gestionar habitaciones y tarifas	Crear y actualizar tipos de habitación, habitaciones, estados (incluido mantenimiento) y tarifas con vigencia.	Administrador	Inventario y tarifas actualizados y aplicados a las nuevas consultas (CU-11).
RF12	Consultar historial de reservas	Buscar reservas por código, huésped, habitación o fechas y mostrar su estado y movimientos.	Recepcionista, administrador	Listado de reservas con su bitácora de cambios (CU-12).
RF13	Generar reporte de ocupación	Calcular, para un periodo, las habitaciones ocupadas, reservadas y disponibles y el porcentaje de ocupación.	Administrador	Reporte de ocupación coherente con las reservas registradas (CU-13).
RF14	Gestionar usuarios	Crear, activar o desactivar cuentas del personal y asignarles el perfil.	Administrador	Usuarios registrados con perfil y estado (CU-14).
RF15	Aplicar penalidad por cancelación tardía	Calcular y registrar la penalidad cuando la cancelación ocurre fuera del plazo definido por la política del hotel.	Recepcionista	Penalidad calculada, reflejada en el pago y anotada en la bitácora (CU-15).

Nota. Elaboración propia. El código entre paréntesis en la última columna indica el caso de uso de la sección 5 que materializa cada requerimiento.

Los requerimientos RF03, RF04, RF05 y RF06 forman el núcleo transaccional del sistema: la consulta de disponibilidad alimenta la creación de la reserva, y esta solo se considera completa cuando el pago y el comprobante quedan registrados en la misma unidad de trabajo. Los requerimientos RF07 a RF10 cubren los cambios de estado posteriores y, junto con RF12, dependen de la bitácora para dejar evidencia de cada operación. Los requerimientos RF11 y

RF14 corresponden a la administración del inventario y de las cuentas, y RF13 explota los datos registrados para apoyar las decisiones del administrador.

10. REQUERIMIENTOS NO FUNCIONALES

Los requerimientos no funcionales establecen las condiciones de calidad bajo las cuales deben prestarse los servicios anteriores. Para organizarlos se toma como referencia el modelo de calidad del producto de la norma ISO/IEC 25010, que agrupa las características en seguridad, eficiencia de desempeño, usabilidad, fiabilidad, mantenibilidad, compatibilidad y portabilidad (ISO/IEC, 2011). A ellas se añade la trazabilidad, que en un sistema transaccional constituye una exigencia del negocio. Cada requerimiento de la Tabla 3 incluye un criterio verificable, de modo que pueda comprobarse en las pruebas de aceptación de las etapas siguientes.

Tabla 3 Requerimientos no funcionales del sistema Hotel Reservas

Código	Atributo	Requerimiento	Criterio verificable
RNF01	Seguridad	Todo acceso a la API debe autenticarse mediante un token JWT emitido al iniciar sesión, y cada operación debe autorizarse según el perfil del usuario. Las contraseñas se almacenan únicamente como hash.	Ninguna solicitud sin token válido obtiene datos; un recepcionista no puede ejecutar RF11 ni RF14; en la base de datos no existe ninguna contraseña en texto claro.
RNF02	Integridad de los datos	La confirmación, modificación y cancelación de reservas deben ejecutarse como transacciones ACID; no pueden existir dos reservas activas solapadas para la misma habitación.	En una prueba con dos sesiones simultáneas sobre la última habitación, exactamente una reserva queda confirmada y la otra recibe el código HTTP 409; tras cualquier fallo no quedan pagos ni comprobantes sin reserva.
RNF03	Rendimiento	Las consultas de disponibilidad y la confirmación de reservas deben responder en tiempos adecuados para la atención en recepción.	Con 25 habitaciones y hasta 10 usuarios concurrentes, la consulta de disponibilidad responde en menos de 2 segundos y la confirmación en menos de 3 segundos en el 95 % de los casos.
RNF04	Disponibilidad	El servicio debe estar accesible durante el horario de operación del hotel, incluidas las noches y los fines de semana.	Disponibilidad mensual igual o superior al 99 % medida sobre la API; las ventanas de mantenimiento se programan y se anuncian con anticipación.

Código	Atributo	Requerimiento	Criterio verificable
RNF05	Usabilidad	La aplicación móvil debe permitir completar las tareas frecuentes con pocos pasos y mensajes claros, sin capacitación extensa.	Un recepcionista nuevo crea una reserva completa (RF03 a RF06) en un máximo de cinco pantallas y en menos de tres minutos; los mensajes de error indican la causa y la acción sugerida.
RNF06	Mantenibilidad	La solución debe conservar la separación en capas definida en la sección 4, de modo que los cambios en una capa no obliguen a modificar las demás.	Las reglas de negocio residen solo en la API; una modificación de la interfaz móvil no requiere cambios en la base de datos; el código cuenta con pruebas unitarias para los servicios de reservas.
RNF07	Compatibilidad	La aplicación debe ejecutarse en los dispositivos Android del personal y la API debe poder consumirse desde otros clientes.	El APK funciona en Android 10 o superior; la API expone sus recursos en JSON sobre HTTPS y puede probarse desde Postman sin la aplicación.
RNF08	Trazabilidad	Cada cambio de estado de una reserva debe registrar el usuario responsable, la fecha, el estado anterior y el nuevo.	Para cualquier reserva es posible reconstruir su historial completo desde la bitácora, y ninguna anotación carece de usuario o de fecha.

Nota. Elaboración propia con base en las características de calidad de ISO/IEC 25010 (ISO/IEC, 2011).

Los requerimientos RNF02 y RNF08 responden directamente a la problemática de la Fase I, pues son los que impiden la sobreventa y aportan la evidencia para atender reclamaciones. RNF01 protege los datos personales de los huéspedes, cuya recolección y tratamiento se rigen en Colombia por la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012). Los demás requerimientos aseguran que la solución sea utilizable y sostenible en un hotel pequeño o mediano, donde no se cuenta con personal técnico permanente.

11. MARCO CONCEPTUAL

Esta sección expone los conceptos que fundamentan el proyecto y los relaciona con las funciones descritas. Se organiza en cuatro grupos: los conceptos propios del dominio hotelero, los conceptos de sistemas transaccionales, los conceptos tecnológicos de la plataforma elegida y, por último, la forma en que todos ellos se aplican al prototipo.

11.1 CONCEPTOS FUNDAMENTALES DEL PROYECTO

La reserva es el compromiso mediante el cual el hotel aparta una habitación para un huésped durante un periodo determinado, a cambio de un pago o de una garantía. En el sistema se representa como la entidad Reserva y pasa por los estados pendiente, confirmada, en casa, cerrada y cancelada, que describen su ciclo de vida desde la solicitud hasta la salida del huésped. La disponibilidad es la condición de una habitación que no está en mantenimiento ni tiene reservas activas cuyas fechas se solapen con el periodo consultado; su cálculo correcto es el requisito previo de toda reserva. La sobreventa, conocida en el sector como overbooking, se produce cuando se aceptan más reservas que habitaciones existen para una misma fecha; en este proyecto se trata como un error que debe impedirse, no como una práctica comercial.

El inventario de habitaciones agrupa las unidades físicas del hotel por tipo y capacidad, y la tarifa fija el precio por noche de cada tipo con un periodo de vigencia, lo que permite conservar el historial de precios. El comprobante es el soporte documental entregado al huésped al confirmar la reserva y constituye la evidencia principal ante una reclamación. El check-in y el check-out son los registros de llegada y salida que vinculan la reserva con la ocupación efectiva de la habitación. Estos conceptos definen el vocabulario con el que se nombraron los casos de uso, las clases y las tablas en las secciones anteriores.

11.2 CONCEPTOS DE SISTEMAS TRANSACCIONALES

Un sistema de procesamiento de transacciones en línea (OLTP) es aquel que registra las operaciones del negocio en el momento en que ocurren, atendiendo muchas solicitudes cortas y concurrentes sobre una base de datos compartida (Silberschatz et al., 2020). Una transacción es una secuencia de operaciones que se ejecuta como una unidad lógica de trabajo. Haerder y Reuter (1983) sintetizaron sus propiedades en el acrónimo ACID: atomicidad, porque la transacción se completa en su totalidad o no deja ningún efecto; consistencia, porque lleva la

base de datos de un estado válido a otro estado válido; aislamiento, porque las transacciones concurrentes no observan los resultados intermedios de las demás; y durabilidad, porque los efectos confirmados sobreviven a fallos posteriores. Gray y Reuter (1993) explican que estas garantías son las que permiten que varios usuarios operen al mismo tiempo sobre los mismos datos sin corromperlos.

La concurrencia es la ejecución simultánea de varias transacciones, y el control de concurrencia es el conjunto de mecanismos que garantiza que el resultado sea equivalente a alguna ejecución en serie. En el enfoque pesimista el motor bloquea los registros que una transacción va a leer o modificar hasta que esta termina; en el enfoque optimista cada registro lleva una marca de versión y la actualización solo se acepta si la versión no cambió desde la lectura (Elmasri y Navathe, 2016). El nivel de aislamiento define cuánto se protege una transacción de los efectos de las demás; el nivel serializable es el más estricto y es el que elimina las anomalías que producirían una doble reserva (Microsoft, 2024f). La integridad referencial, mantenida mediante claves foráneas y restricciones, impide que existan pagos, comprobantes o anotaciones sin una reserva válida. Por último, la bitácora o registro de auditoría conserva la secuencia de cambios con su autor y su momento, lo que hace posible reconstruir lo ocurrido y atender reclamaciones.

11.3 CONCEPTOS TECNOLÓGICOS

.NET MAUI es el marco de Microsoft para construir aplicaciones nativas multiplataforma en C# y XAML a partir de un único código base; en este proyecto se utiliza para generar la aplicación Android distribuida como APK (Microsoft, 2024c). ASP.NET Core Web API permite exponer servicios HTTP que reciben y devuelven datos en formato JSON (Microsoft, 2024e), siguiendo el estilo arquitectónico REST, en el que cada recurso del negocio

se identifica con una ruta y se manipula con los métodos estándar del protocolo (Fielding, 2000). Entity Framework Core es un mapeador objeto-relacional que traduce las clases del dominio en tablas y las operaciones sobre objetos en instrucciones SQL, además de administrar el alcance de las transacciones (Microsoft, 2024d).

Microsoft SQL Server es el sistema de gestión de bases de datos relacional elegido para el almacenamiento; ofrece transacciones explícitas, niveles de aislamiento configurables, bloqueos de fila y el tipo de dato ROWVERSION, que son los instrumentos con los que se materializa el control de concurrencia descrito en la sección 7.2 (Microsoft, 2024a). El token JWT es un formato estándar para transportar, de manera firmada, la identidad y el perfil de un usuario autenticado en cada solicitud a la API. Finalmente, los servicios de inteligencia artificial en la nube, como Azure OpenAI Service, ponen a disposición modelos de lenguaje que interpretan y producen texto en lenguaje natural a través de una API, sin que el equipo deba entrenar ni alojar el modelo (Microsoft, 2024g).

11.4 APLICACIÓN AL PROYECTO

Los conceptos anteriores se combinan en el prototipo de la siguiente manera. El vocabulario del dominio hotelero define las entidades y los estados que el sistema registra. Las propiedades ACID se aplican a la confirmación de reservas, que agrupa en una sola transacción la verificación de disponibilidad, la creación de la reserva, el registro del pago, la emisión del comprobante y la anotación en la bitácora. El control de concurrencia, ya sea mediante bloqueos o mediante versiones, resuelve el caso en que dos recepcionistas intentan asignar la misma habitación. La integridad referencial y las restricciones de la base de datos protegen la coherencia de los registros, y la bitácora cumple el requerimiento de trazabilidad. La arquitectura en capas distribuye estos conceptos entre la aplicación móvil, la API y el motor de datos, y el

servicio de inteligencia artificial, descrito en la sección 13, se apoya en la API para interpretar consultas sin intervenir en la transacción crítica.

12. MARCO CONTEXTUAL

12.1 CONTEXTO DEL PROYECTO

El proyecto se sitúa en los hoteles pequeños y medianos, entendidos como establecimientos con un número reducido de habitaciones, administrados por su propietario o por un equipo pequeño, y con personal de recepción que rota por turnos. En este tipo de organización no suele existir un área de tecnología ni un sistema de gestión hotelera contratado, de modo que la operación diaria se apoya en herramientas de propósito general. El entorno de trabajo es la recepción del hotel, pero también los pasillos, los pisos y las zonas comunes, pues el personal se desplaza constantemente; por ello la solución se concibe como una aplicación móvil. El proyecto se desarrolla en el marco de la asignatura Sistemas Transaccionales del programa de Ingeniería de Software de la Universidad Manuela Beltrán, lo que orienta su énfasis hacia el procesamiento confiable de operaciones concurrentes.

12.2 PROBLEMÁTICA IDENTIFICADA

En la Fase I se estableció que estos hoteles administran las reservas mediante cuadernos, hojas de cálculo compartidas, llamadas y mensajería instantánea. La información queda repartida entre personas y turnos sin una fuente única, lo que origina cuatro situaciones recurrentes: habitaciones reservadas dos veces para las mismas fechas, solicitudes que se pierden en el cambio de turno, diferencias entre lo reservado, lo pagado y lo facturado, y ausencia de evidencia para resolver reclamaciones. El punto crítico es el momento en que dos personas confirman una reserva al mismo tiempo sin un mecanismo que las coordine; cualquiera de las herramientas

mencionadas permite que ambas operaciones se completen y que el conflicto se descubra solo cuando el huésped llega.

12.3 USUARIOS O BENEFICIARIOS

El sistema tiene dos usuarios directos. El recepcionista atiende a los clientes y necesita consultar disponibilidad y confirmar, modificar o cancelar reservas de manera rápida, desde cualquier lugar del hotel y con la seguridad de que lo que ve corresponde a la situación real. El administrador mantiene el inventario de habitaciones, las tarifas y las cuentas del personal, y necesita informes de ocupación confiables para tomar decisiones sobre precios y mantenimiento. Los beneficiarios indirectos son los huéspedes, que reciben una confirmación cierta y un comprobante, y el propietario del hotel, que reduce las pérdidas por sobreventa y por habitaciones no vendidas. Los usuarios directos tienen formación operativa y no técnica, por lo que la interfaz debe ser sencilla y los mensajes claros.

12.4 NECESIDAD QUE SE PRETENDE ATENDER

La necesidad es contar con un registro único, centralizado y concurrente de la disponibilidad de habitaciones, en el que las acciones de confirmar, modificar y cancelar se realicen de forma atómica, aislada y durable, se puedan ejecutar desde dispositivos móviles y dejen constancia de quién hizo cada cambio y cuándo. No se trata únicamente de digitalizar el cuaderno de reservas: la mejora consiste en que el propio sistema impida la doble reserva en lugar de confiar en la atención del personal, y en que el pago y el comprobante queden ligados a la reserva desde el momento de su confirmación.

12.5 DESCRIPCIÓN DEL PRODUCTO O PROTOTIPO

Hotel Reservas es una aplicación móvil Android, distribuida como APK, conectada a una API y a una base de datos central. Su propósito es administrar el ciclo completo de las reservas de un hotel garantizando la integridad y la trazabilidad de la información. La aplicación gestiona cinco grupos de datos: los usuarios del personal con su perfil, los huéspedes, el inventario de habitaciones con sus tipos y tarifas, las reservas con sus pagos y comprobantes, y la bitácora de cambios. Sus funcionalidades principales, enumeradas en la Tabla 2, son la autenticación, el registro de huéspedes, la consulta de disponibilidad, la creación, modificación y cancelación de reservas, el registro de pagos y comprobantes, el check-in y el check-out, la gestión del inventario y de los usuarios, el historial de reservas y el reporte de ocupación. Desde el punto de vista del usuario, el flujo típico consiste en consultar la disponibilidad de un tipo de habitación para unas fechas, seleccionar una habitación, asociar al huésped, registrar el pago y recibir el comprobante en una sola secuencia de pantallas. El prototipo de la entrega S7 contiene la pantalla de inicio y la navegación entre módulos; las siguientes entregas la conectarán con la API y la base de datos.

12.6 ALCANCE DEL PROYECTO

Durante la etapa académica se desarrollará un sistema para un solo hotel, con un inventario central de habitaciones, dos perfiles de usuario y una habitación por reserva, que cubra el ciclo completo de la reserva y los reportes básicos de ocupación. Los entregables previstos son la base de datos en SQL Server con sus scripts, la API REST con la transacción de confirmación y el control de concurrencia, la aplicación móvil conectada a la API con los casos de uso priorizados, y la demostración de dos usuarios concurrentes sobre la última habitación disponible. Quedan fuera del alcance la integración con pasarelas de pago, con gestores de canales de venta y con plataformas contables, la reserva en línea por parte del huésped y la

operación de varios hoteles. El servicio de inteligencia artificial de la sección 13 se documenta y se diseña conceptualmente en esta fase, pero su implementación se considera una ampliación posterior al núcleo transaccional.

13. MARCO INTEGRADOR: INCORPORACIÓN DEL SERVICIO DE INTELIGENCIA ARTIFICIAL

13.1 SERVICIO DE INTELIGENCIA ARTIFICIAL PROPUESTO

Se propone incorporar un asistente de consultas en lenguaje natural basado en Azure OpenAI Service, el servicio de Microsoft que ofrece acceso, mediante una API, a modelos de lenguaje de gran tamaño como los de la familia GPT-4o (Microsoft, 2024g). La elección se justifica por tres razones: el servicio se integra con la misma plataforma en la que se construyen la API y la base de datos, lo que simplifica la autenticación y el despliegue; dispone de la capacidad de llamada a funciones, que permite al modelo traducir una pregunta en una invocación estructurada a una función definida por el equipo (Microsoft, 2024h); y su política de datos establece que la información enviada no se usa para entrenar los modelos (Microsoft, 2024i), condición necesaria para un sistema que maneja datos de huéspedes.

13.2 FUNCIÓN DENTRO DEL PROYECTO

El asistente atenderá preguntas que hoy exigen varios pasos en la interfaz, formuladas tal como el personal las expresa en la recepción: qué habitaciones dobles hay libres del 10 al 12 de octubre, cuánto costaría una suite por tres noches en diciembre, en qué estado está la reserva de un huésped determinado o cuántas llegadas hay programadas para mañana. El modelo no responde con su propio conocimiento: interpreta la pregunta, identifica la función del sistema que puede responderla y los parámetros necesarios, y la API ejecuta esa función sobre el

repositorio central. Con el resultado real, el modelo redacta la respuesta en lenguaje natural. La necesidad que ayuda a resolver es la de agilizar la atención al cliente y reducir los errores de interpretación de fechas y tipos de habitación, sin alterar el control transaccional.

13.3 INTEGRACIÓN CON EL SISTEMA

El servicio se ubica en la capa de lógica de negocio, como un componente adicional de la API, de modo que la aplicación móvil solo agrega una pantalla de conversación y la base de datos no sufre cambios. La Figura 5 muestra el flujo conceptual. El recepcionista escribe la consulta, la aplicación la envía a la API con su token JWT, la API la remite al servicio de inteligencia artificial junto con la descripción de las funciones permitidas, el servicio devuelve la función y los parámetros que considera adecuados, la API ejecuta esa función reutilizando los mismos servicios del negocio de los casos de uso CU-03 y CU-12, y el resultado se envía de nuevo al servicio para que redacte la respuesta. Tres reglas de diseño delimitan la integración: el asistente solo accede a funciones de lectura, de manera que nunca crea, modifica ni cancela reservas; la confirmación (CU-04) se realiza siempre desde la pantalla transaccional habitual, con la verificación y el bloqueo descritos en la sección 7.2; y a la API externa se envían únicamente los datos necesarios para la consulta, sin documentos de identidad ni datos de contacto de los huéspedes.

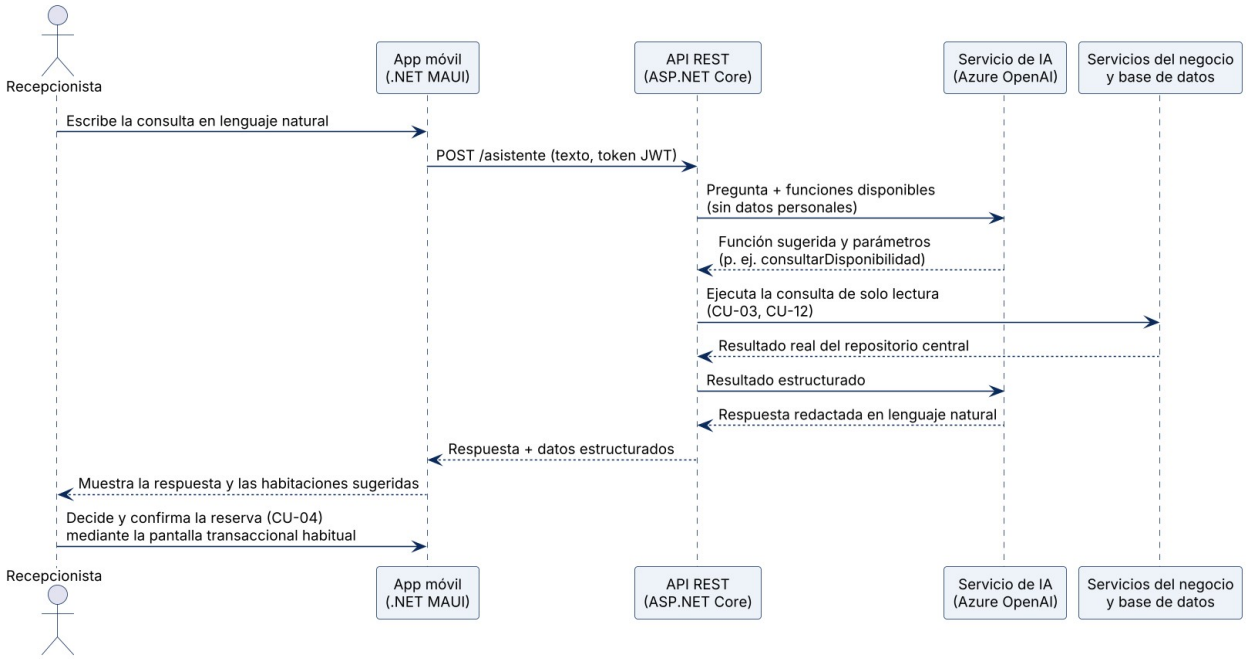


Figura 5 Flujo conceptual del asistente de consultas en lenguaje natural

Nota. Elaboración propia. El servicio de inteligencia artificial interpreta y redacta; la API ejecuta las consultas y conserva el control de las transacciones.

Tabla 4 Ejemplos de consultas y funciones que las atienden

Consulta del usuario	Función de la API invocada	Caso de uso	Resultado
¿Qué habitaciones dobles hay libres del 10 al 12 de octubre?	consultarDisponibilidad(tipo, entrada, salida)	CU-03	Lista de habitaciones disponibles con tarifa y valor estimado.
¿Cuánto costaría una suite por tres noches desde el 20 de diciembre?	consultarDisponibilidad y calcularTotal(noches)	CU-03	Valor estimado según la tarifa vigente del tipo de habitación.
¿En qué estado está la reserva RSV000000045?	consultarReserva(codigo)	CU-12	Estado actual de la reserva y último movimiento de la bitácora.
¿Cuántas llegadas tenemos programadas para mañana?	consultarLlegadas(fecha)	CU-12, CU-13	Número de reservas confirmadas con entrada en la fecha indicada.
Reserva la 305 para Laura Martínez del 3 al 5	No se ejecuta; el asistente indica que la confirmación se realiza en la pantalla de reservas	CU-04 (solo por la interfaz transaccional)	Mensaje de orientación; la transacción conserva su control humano y su bloqueo.

Nota. Elaboración propia. Los nombres de las funciones corresponden a operaciones de ServicioReservas y de los repositorios definidos en la Fase II.

13.4 RESULTADO ESPERADO

El beneficio esperado para el recepcionista es responder a un cliente en una sola interacción, con datos tomados del repositorio central, en lugar de navegar por varias pantallas y filtros; para el administrador, obtener de manera inmediata cifras que hoy requieren consultar el reporte de ocupación. Se espera además reducir los errores por interpretación de fechas, pues el asistente las normaliza antes de invocar la función. Para el proceso, el resultado es una mejora en la rapidez de atención que no compromete la integridad: el asistente consulta, pero la transacción crítica sigue bajo el control de la API y del motor de datos. El éxito de la propuesta podrá medirse con el tiempo medio de respuesta a una consulta de disponibilidad y con la proporción de consultas resueltas correctamente en pruebas con usuarios.

13.5 CONSIDERACIONES Y LÍMITES

La propuesta reconoce tres riesgos y la manera de mitigarlos. El primero es que el modelo produzca respuestas no fundamentadas; se mitiga al obligarlo a responder únicamente a partir del resultado de las funciones, sin acceso directo a la base de datos. El segundo es la protección de los datos personales de los huéspedes, atendida con la regla de enviar solo la información indispensable y con la política de no entrenamiento del servicio (Microsoft, 2024i), en coherencia con la Ley 1581 de 2012. El tercero es el costo por consumo del servicio, que se controla limitando el uso al personal autenticado y registrando cada consulta en la bitácora del sistema. Conforme a la guía de esta actividad, en la presente fase el servicio se documenta y se

diseña conceptualmente; su conexión efectiva con la API se evaluará una vez construido el núcleo transaccional.

CONCLUSIONES

El trabajo de análisis y diseño de esta etapa convirtió la propuesta formulada en la Fase I en una solución organizada. El objetivo general responde directamente al problema y los cinco objetivos específicos se desarrollan dentro del documento. La arquitectura concentra las transacciones en el servidor y la base de datos; el modelo de casos de uso reúne quince funciones distribuidas entre dos actores y sitúa la confirmación de reservas como proceso central; el diagrama de clases estructura esas funciones por medio de las entidades, relaciones y operaciones del dominio; el modelo relacional las traduce en nueve tablas de SQL Server con claves y reglas destinadas a impedir superposiciones; y la matriz demuestra la correspondencia entre las decisiones de diseño y las necesidades detectadas. En síntesis, la prevención de reservas duplicadas queda incorporada a la solución técnica y deja de depender únicamente del cuidado de los colaboradores.

La Fase III completó esa base con los elementos que hacen verificable el diseño. Los quince requerimientos funcionales reproducen uno a uno los casos de uso, los ocho requerimientos no funcionales fijan criterios comprobables de seguridad, integridad, rendimiento, disponibilidad, usabilidad, mantenibilidad, compatibilidad y trazabilidad, y los marcos conceptual y contextual explican los fundamentos transaccionales y el entorno hotelero en que se usará la aplicación. El marco integrador propone un asistente de consultas en lenguaje natural con Azure OpenAI Service, limitado a funciones de lectura, de modo que agiliza la atención sin intervenir en la transacción crítica. Las etapas siguientes deberán implementar la base de datos con sus scripts, construir la API con la transacción de confirmación y decidir con pruebas entre el bloqueo pesimista y la concurrencia optimista, conectar la aplicación móvil

comenzando por CU-01, CU-03 y CU-04, verificar los criterios de la Tabla 3 mediante pruebas de aceptación y, por último, evaluar la conexión del asistente de inteligencia artificial.

REFERENCIAS

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Congreso de la República de Colombia. (2012). Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. *Diario Oficial*, 48587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Gray, J. y Reuter, A. (1993). *Transaction processing: Concepts and techniques*. Morgan Kaufmann.

Haerder, T. y Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys, 15*(4), 287-317. https://doi.org/10.1145/289.291

ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models*. International Organization for Standardization. https://www.iso.org/standard/35733.html

Microsoft. (2024a). *Transacciones (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/t-sql/language-elements/transactions-transact-sql

Microsoft. (2024b). *Model-View-ViewModel (MVVM)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/architecture/maui/mvvm

Microsoft. (2024c). *¿Qué es .NET MAUI?* Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/maui/what-is-maui

Microsoft. (2024d). *Información general de Entity Framework Core*. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/

Microsoft. (2024e). *Creación de API web con ASP.NET Core*. Microsoft Learn. https://learn.microsoft.com/es-es/aspnet/core/web-api/

Microsoft. (2024f). *SET TRANSACTION ISOLATION LEVEL (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/t-sql/statements/set-transaction-isolation-level-transact-sql

Microsoft. (2024g). *¿Qué es Azure OpenAI Service?* Microsoft Learn. https://learn.microsoft.com/es-es/azure/ai-services/openai/overview

Microsoft. (2024h). *Cómo usar la llamada a funciones con Azure OpenAI Service*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/ai-services/openai/how-to/function-calling

Microsoft. (2024i). *Datos, privacidad y seguridad de Azure OpenAI Service*. Microsoft Learn. https://learn.microsoft.com/es-es/legal/cognitive-services/openai/data-privacy

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Pressman, R. S. y Maxim, B. R. (2021). *Ingeniería del software: un enfoque práctico* (9.ª ed.). McGraw-Hill.

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.